RNA-seq Data Analysis: Sample Comparison Using PCA, Heatmap, and UPGMA Clustering

2026-01-25

Introduction

RNA-seq experiments generate large gene expression datasets that allow researchers to explore transcriptional differences between biological samples.

In this study, raw RNA-seq count data obtained from the **GEO (Gene Expression Omnibus)** database were analyzed to investigate global gene expression patterns across samples.

The analysis workflow includes several basic exploratory steps commonly used in RNA-seq analysis:

- Filtering low-expression genes
- Log transformation of count data
- Principal Component Analysis (PCA) to explore sample variability
- Heatmap visualization of highly variable genes
- Hierarchical clustering using the UPGMA method

The main objective of this analysis is to demonstrate **basic exploratory RNA-seq analysis techniques using a manual and transparent workflow in R**.

This tutorial focuses on helping students understand the logic behind each step rather than relying on automated pipelines.

Dataset Description

The RNA-seq dataset used in this study was obtained from the **GEO database**.

Dataset details:

- **Source:** GEO (Gene Expression Omnibus)
- **File:** `GSE164073_raw_counts_GRCh38.p13_NCBI.tsv.gz`
- **Content:** Gene-level raw count matrix
- **Dimensions:** 39,376 genes $\times$ 18 samples

The dataset contains RNA-seq gene expression counts from **human (Homo sapiens)** samples.

Each row represents a gene and each column represents a sample.
The values in the matrix correspond to **raw read counts**, which indicate how many sequencing reads were mapped to each gene.

Data Preprocessing

Before performing downstream analysis, RNA-seq count data must be prepared and formatted correctly.

In this step we will:

- read the raw count file downloaded from GEO
- inspect the structure and dimensions of the dataset
- prepare the count matrix for downstream analysis

These preprocessing steps ensure that the data are properly formatted for exploratory analysis such as **PCA, heatmap visualization, and clustering**.

```r
# File name
file <- "GSE164073_raw_counts_GRCh38.p13_NCBI.tsv.gz"  #Downloaded from the GEO database


# Read the file
counts_tbl <- read.delim(gzfile(file), check.names = FALSE)

# Check dimensions
dim(counts_tbl)
```

```
## [1] 39376    19
```

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a widely used dimensionality reduction technique that helps summarize large gene expression datasets.

RNA-seq data typically contain **thousands of genes**, making it difficult to directly visualize relationships between samples. PCA simplifies the dataset by projecting the samples onto a smaller number of components that capture the largest sources of variation.

In this analysis:

- low-expression genes are filtered to reduce noise
- count values are log-transformed to stabilize variance
- the data are scaled to ensure genes contribute equally
- PCA is applied to explore global variation between samples

The resulting PCA plot allows us to visualize how samples relate to each other based on their overall gene expression profiles.

```r
# Extract gene IDs from the first column
gene_id <- counts_tbl[[1]]

# Convert count data (excluding first column) into a numeric matrix
counts <- as.matrix(counts_tbl[, -1])

# Assign gene IDs as row names of the matrix
rownames(counts) <- gene_id


# Check the dimensions of the count matrix (genes × samples)
dim(counts)
```

```
## [1] 39376    18
```

```r
# Display the first 3 genes and first 3 samples
counts[1:3, 1:3]
```

```
##           GSM4996084 GSM4996085 GSM4996086
## 100287102          2          5          3
## 653635           244        236        337
## 102466751         25         17         34
```

```r
# Filter out low-expression genes
# Keep genes with total counts greater than 10 across all samples
keep <- rowSums(counts) > 10

# Create a filtered count matrix
counts_f <- counts[keep, , drop = FALSE]


# Check dimensions after filtering
dim(counts_f)
```

```
## [1] 23484    18
```

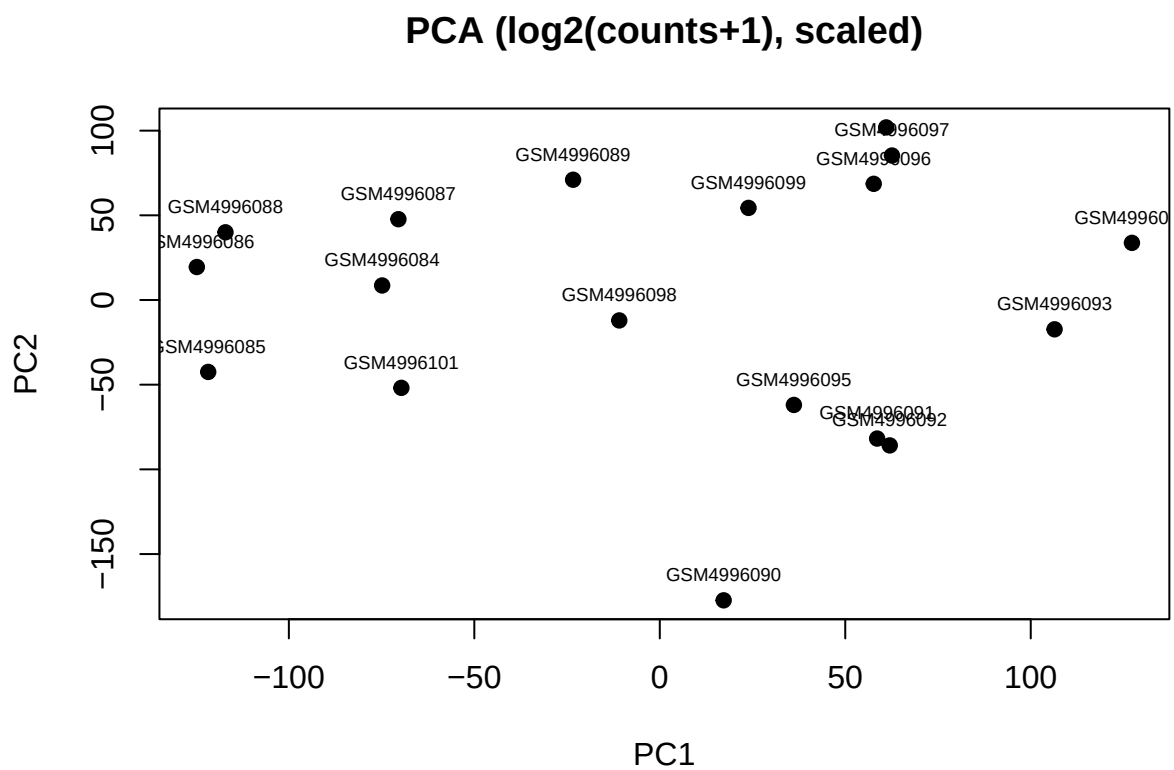
```r
# Apply log2 transformation to stabilize variance
# Adding +1 prevents log(0) errors
expr_log <- log2(counts_f + 1)


# Scale and transpose the data for PCA
# Samples become rows, genes become columns
expr_scaled <- scale(t(expr_log))


# Perform Principal Component Analysis (PCA)
pca <- prcomp(expr_scaled)
```

```r
# Plot the first two principal components
plot(pca$x[,1], pca$x[,2],
     pch = 19,                          # Solid circle points
     xlab = "PC1",                      # X-axis label
     ylab = "PC2",                      # Y-axis label
     main = "PCA (log2(counts+1), scaled)")  # Plot title


# Add sample labels to each point
text(pca$x[,1], pca$x[,2],
     labels = colnames(counts_f),       # Sample names
     pos = 3,                           # Position above points
     cex = 0.6)                         # Text size
```



How to Read This PCA Plot

The PCA plot summarizes the global gene expression patterns of all RNA-seq samples using the first two principal components (**PC1** and **PC2**).

These components capture the **largest sources of variation in the dataset**.

Important points when interpreting the PCA plot:

- **Each dot represents one RNA-seq sample.**
- **Distance between dots reflects similarity in gene expression.**
  Samples that are closer together have more similar transcriptomic profiles.
- **Samples that are far apart show distinct expression patterns.**
- **Clusters of points may indicate biological similarity** between samples.

Overall, PCA helps us quickly visualize how samples group together based on their global gene expression profiles and can reveal **biological variation or sample heterogeneity**.

Enhanced Visualization of PCA Results

To improve visualization, the PCA results can be converted into a structured data frame containing the coordinates of each sample in the PCA space.

This allows us to generate a clearer scatter plot with:

- larger points
- readable sample labels
- background grid lines for easier interpretation

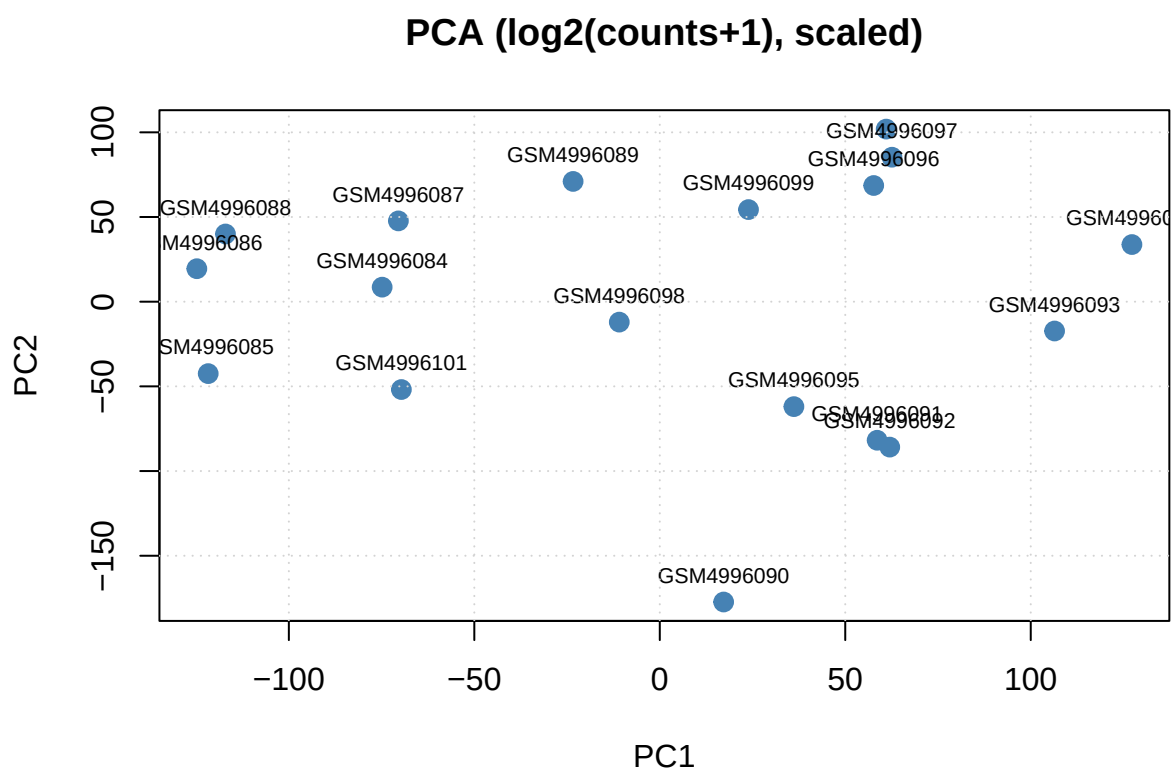
Improved visualization helps make PCA plots **more informative and easier to interpret**, especially when presenting results in reports or publications.

```r
# Convert PCA scores into a data frame
# Each row represents one sample
# PC1 and PC2 are the first two principal components
pca_df <- data.frame(
  sample = rownames(pca$x),   # Sample names
  PC1 = pca$x[,1],            # Scores for Principal Component 1
  PC2 = pca$x[,2]             # Scores for Principal Component 2
)


# Create an improved PCA scatter plot using base R
plot(pca_df$PC1, pca_df$PC2,
     pch = 19,                # Solid circle points
     cex = 1.3,               # Point size
     col = "steelblue",       # Point color
     xlab = "PC1",            # X-axis label
     ylab = "PC2",            # Y-axis label
     main = "PCA (log2(counts+1), scaled)")  # Plot title


# Add sample labels next to each point
text(pca_df$PC1, pca_df$PC2,
     labels = pca_df$sample,  # Sample names
     pos = 3,                 # Position above points
     cex = 0.7)               # Label size
```

```r
# Add a background grid for better readability
grid()
```



Heatmap

Heatmaps are commonly used in RNA-seq analysis to visualize patterns of gene expression across samples.

Instead of displaying all genes, it is often more informative to focus on **highly variable genes**, because these genes contribute the most to differences between samples.

In this step:

- the variance of each gene across samples is calculated
- the **top 50 most variable genes** are selected
- a heatmap is generated to visualize expression patterns

Heatmaps allow us to observe **global patterns of gene expression** and identify groups of samples with similar transcriptional profiles.

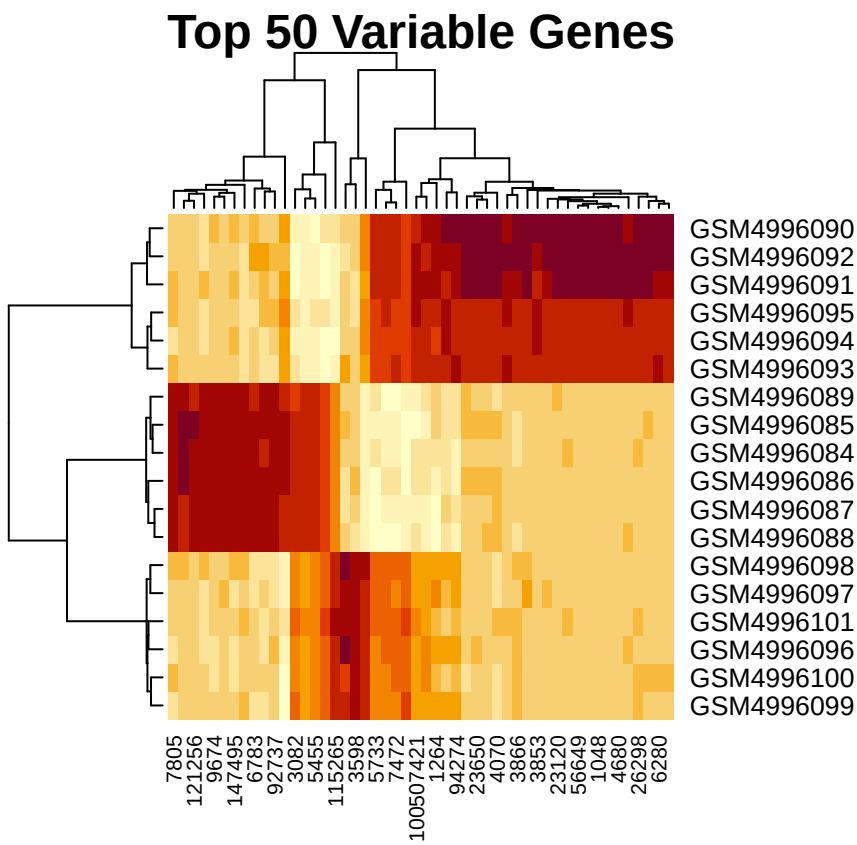
```r
# Calculate variance for each gene across all samples
# This measures how much each gene's expression changes between samples
gene_var <- apply(expr_log, 1, var)
```

```r
# Select the top 50 genes with the highest variance
# These genes show the strongest expression differences
top <- order(gene_var, decreasing = TRUE)[1:50]


# Create a heatmap of the selected genes
heatmap(

  # Extract expression values of the top variable genes
  # Transpose and scale them for visualization
  scale(t(expr_log[top, ])),

  scale = "none",                 # Disable additional scaling by heatmap()
  main = "Top 50 Variable Genes"  # Title of the heatmap
)
```



How to Read This Heatmap

This heatmap shows the expression patterns of the **top 50 most variable genes** across all RNA-seq samples.

Interpretation guidelines:

- **Rows represent samples**
- **Columns represent genes**
- **Colors represent relative expression levels**

Typical color interpretation:

- **Dark red → high expression**
- **Light/yellow → low expression**

The dendrograms (tree-like structures) represent **hierarchical clustering** based on expression similarity.

The heatmap helps reveal:

- clusters of samples with similar gene expression patterns
- groups of genes that show coordinated upregulation or downregulation
- potential biological differences between sample groups

Heatmap Visualization of Highly Variable Genes

Advanced heatmap visualization can be created using the **pheatmap** package.

Compared to the basic heatmap function, this approach provides additional features such as:

- improved clustering visualization
- sample annotations
- better color scaling

In this example, temporary group labels are added to demonstrate how **sample annotations** can be displayed above the heatmap.

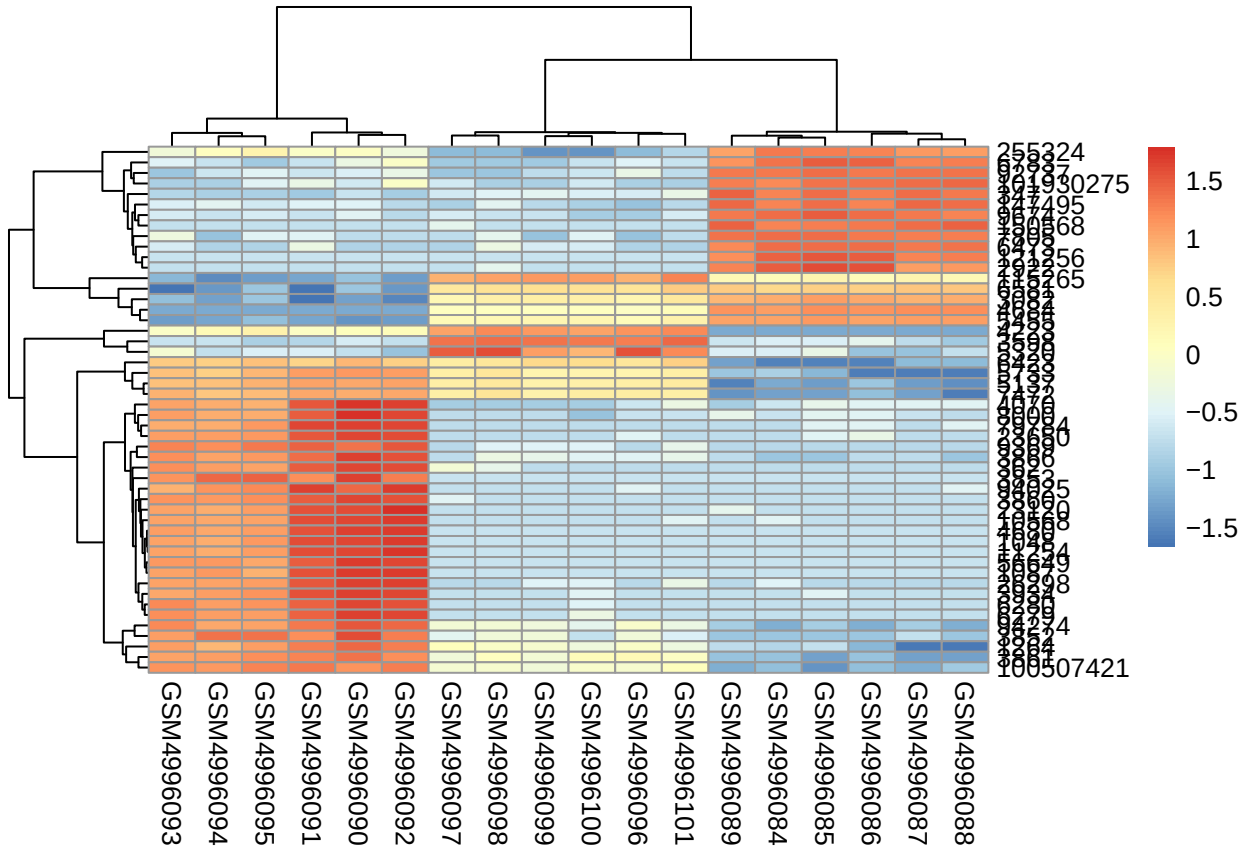
These annotations help visualize how samples from different groups relate to each other in the clustering structure.

```r
# Install and load the pheatmap package for advanced heatmap visualization
# install.packages("pheatmap")
library(pheatmap)


# Extract expression values of the top 50 most variable genes
# Rows = genes, Columns = samples
mat <- expr_log[top, ]


# Apply Z-score normalization to each gene
# This standardizes expression values across samples
mat_s <- t(scale(t(mat)))


# Generate a basic heatmap without sample annotation
pheatmap(mat_s)
```



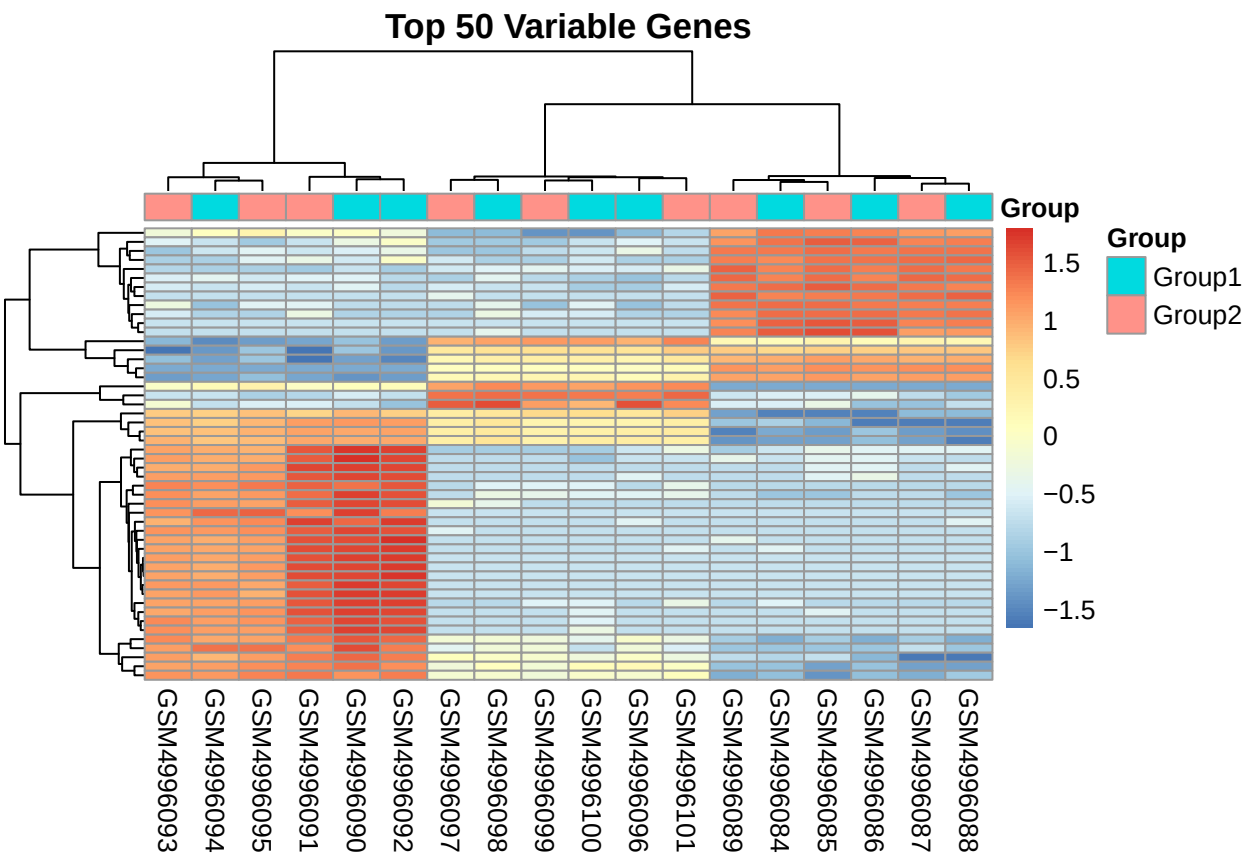

```r
# Extract sample names from the filtered count matrix
samples <- colnames(counts_f)


# Create temporary (dummy) group labels for visualization
# These labels are used only to add color annotations
group <- rep(c("Group1","Group2"), length.out = length(samples))
names(group) <- samples


# Convert group labels into a factor
cols <- as.factor(group)


# Create an annotation data frame for the heatmap
# This will add colored bars above the samples
ann <- data.frame(Group = cols)
rownames(ann) <- colnames(mat_s)


# Generate an annotated heatmap with sample group information
pheatmap(mat_s,
         annotation_col = ann,        # Add group annotation
         show_colnames = TRUE,        # Display sample names
         show_rownames = FALSE,       # Hide gene names for clarity
         main = "Top 50 Variable Genes")  # Heatmap title
```



Difference Between the Two Heatmaps

Two heatmaps are generated in this analysis.

First heatmap

- Displays only gene expression values
- Does not include sample group information
- Useful for observing overall expression patterns

Second heatmap

- Includes sample group annotations
- Uses standardized expression values (z-score)
- Provides a clearer and more structured visualization

The second heatmap is generally more informative because it allows us to observe:

- how samples cluster together
- whether groups separate based on gene expression
- which genes drive the clustering patterns

UPGMA Clustering

Hierarchical clustering is another method used to examine relationships between samples based on their gene expression profiles.

In this analysis we use the **UPGMA method (Unweighted Pair Group Method with Arithmetic Mean)**, which is a form of **average linkage clustering**.

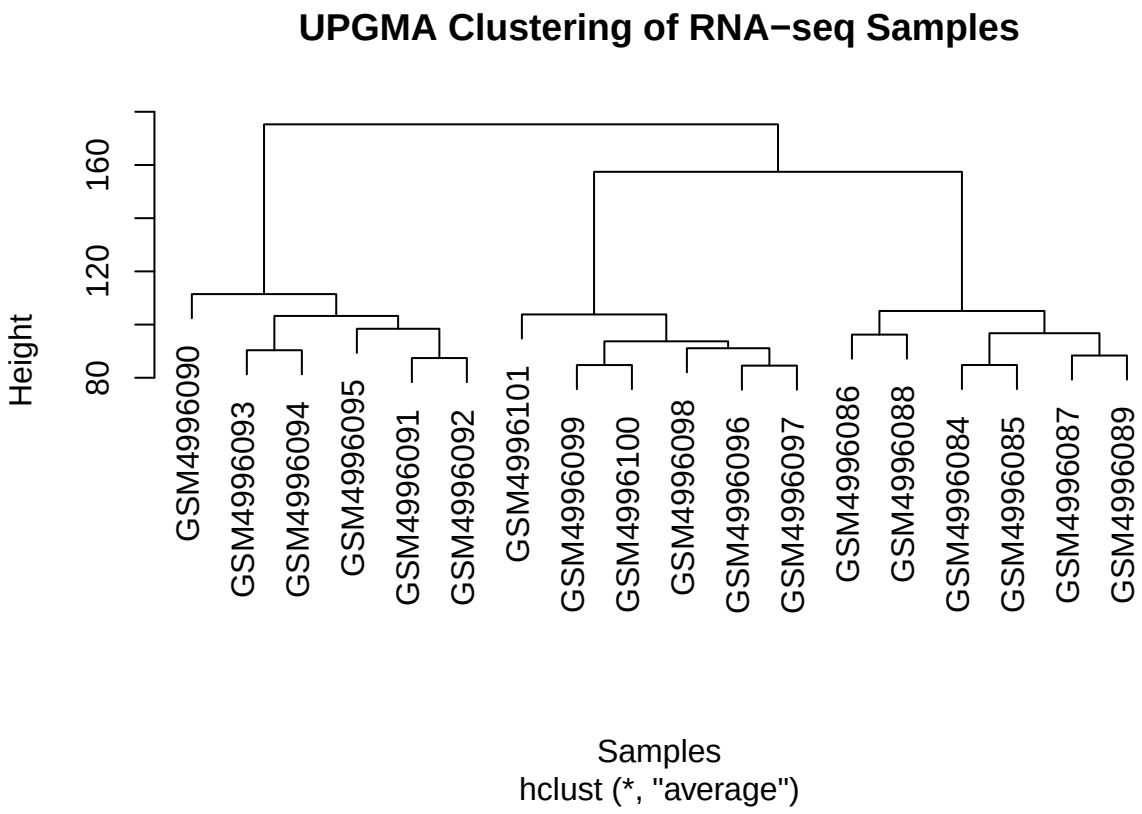
The clustering is performed using a **distance matrix**, where each distance represents how different two samples are based on their gene expression values.

The result is visualized as a **dendrogram**, which shows how samples are grouped hierarchically.

```r
# Calculate the distance matrix between samples
# Each distance represents how different two samples are
# based on their gene expression profiles
dist_matrix <- dist(t(expr_log))


# Perform hierarchical clustering using the UPGMA method
# "average" refers to average linkage clustering
hc <- hclust(dist_matrix, method = "average")


# Plot the hierarchical clustering dendrogram
plot(hc,
     main = "UPGMA Clustering of RNA-seq Samples",
     xlab = "Samples",
     ylab = "Height")
```



How to Read This UPGMA Clustering Plot

The dendrogram displays hierarchical relationships between RNA-seq samples.

Interpretation guidelines:

- **Each label represents one sample.**
- **Samples that join together at lower heights are more similar.**
- **Samples that merge at higher heights are more different.**
- The vertical axis (**height**) represents the distance between clusters.

Hierarchical clustering provides an alternative way to visualize relationships between samples.

What This Analysis Shows

The clustering patterns observed in the dendrogram support the results obtained from the **PCA and heatmap analyses**.

Samples that appear close together in PCA space and show similar expression patterns in the heatmap tend to cluster together in the hierarchical tree as well.

This consistency across multiple visualization methods strengthens confidence in the observed biological patterns within the dataset.